

Two $6k \pm 1$ Factoring Methods: the Wheel Scan and the Small-Factor Sieve

A comparison

Shaimaa Soltan

August 2026

Abstract

Both methods rest on the single fact that, after peeling 2 and 3, every prime factor is of the form $6k \pm 1$. The first, `wheel_scan`, is $6k \pm 1$ *trial division* with early exit: it returns the smallest factor of a single N . The second builds a reusable list P of small factors once, then screens many N against it. We give both, then compare them on completeness, cost, memory, and behaviour on large semiprimes.

1 Method 1 — the wheel scan

Peel 2 and 3; then scan the candidates $M = 6m - 1$ and $M = 6m + 1$ for $m = 1, 2, 3, \dots$ up to $\sqrt{N}$, returning at the first M with $N \bmod M = 0$. Covering *both* residue classes $\{6m - 1, 6m + 1\}$ is essential — a factor may lie in either (e.g. $55 = 5 \cdot 11$ has $N \equiv 1 \pmod 6$ but both primes $\equiv 5$).

```
1 def wheel_scan(N):
2     """Scan M = 6m-1 and 6m+1 for m = 1,2,3,...; EXIT at the first M with N%M==0.
3     Peels 2 and 3 first; returns N if no factor <= sqrt(N) (N is prime)."""
4     for p in (2, 3): # peel 2 and 3
5         if N % p == 0:
6             return p
7     lim = isqrt(N)
8     m = 1
9     while 6 * m - 1 <= lim:
10        for M in (6 * m - 1, 6 * m + 1): # both classes: 5,7 / 11,13 / 17,19 / ...
11            if M <= lim and N % M == 0:
12                return M # early exit at the first m giving mod 0
13        m += 1
14    return N # prime
```

Properties. Deterministic and *complete*: it returns the smallest prime factor of N if one is $\leq \sqrt{N}$, else declares N prime. Cost $O(\sqrt{N})$ in the worst case (smallest factor near $\sqrt{N}$); $O(1)$ memory; no precomputation. Instant when a small factor exists (early exit).

2 Method 2 — the small-factor sieve

Build a set S of $6k \pm 1$ numbers; sweep a range and record, in a list P , every small divisor $k \in S$ of the swept numbers together with the cofactor $x//k$ (*integer* division). Then `N_P` screens any N against P .

```

1 L = {}
2 for x in range(1, 10000, 2):
3     if x % 6 == 5: # what B==8.5 and D==0.5 actually select
4         L[x] = x + 2 # the pair (6k-1, 6k+1)
5
6 L2_Y = [k % v for v in L.values() for k in L.keys()
7         if k != v and v % k == 0 and k % v not in L.values()]
8 L2_X = [v % k for k in L.keys() for v in L.values()
9         if k != v and k % v == 0 and v % k not in L.keys()]
10 S = set(L2_X) | set(L2_Y)
11
12 P = []
13 for x in range(10001, 100000, 6):
14     if x % 2 == 0 or x % 3 == 0:
15         continue
16     for k in S:
17         if (x % k == 0) and k != 1 and k != x:
18             if k not in P: P.append(k)
19             if x // k not in P: P.append(x // k) # // (integer!) -- not /
20
21 def N_P(N):
22     L = [k for k in P if N % k == 0]
23     print(f"Factors of {N}: {L}")

```

Properties. After a one-time $O(\text{range} \times |S|)$ build, each query is only $O(|P|)$ — fast for screening *many* numbers. But it is *partial*: it finds only factors that live in P (small ones), and it lists *composite* divisors too (e.g. $161 = 7 \cdot 23$). It also skips multiples of 3 while building P , so it cannot report the factor 3 itself.

A subtle bug worth flagging. The original used `P.append(x/k)` (true division), putting *floats* like 13763.0 in P . For a large integer N , $N \% 13763.0$ first rounds N to ~ 16 digits, so it returns 0.0 by accident — a *false* factor ($N \bmod 13763 = 6862 \neq 0$). The fix is one character: `x // k`.

3 Comparison

	wheel_scan	sieve (P, N_P)
Strategy	$6k \pm 1$ trial division, early exit	precompute small-factor list, screen N
Peels 2, 3	yes (explicit)	skips them (cannot report 3)
Returns	smallest prime factor $\leq \sqrt{N}$	small divisors in P (incl. composites)
Completeness	complete ($\leq \sqrt{N}$)	partial (only factors in P)
Per- N cost	$O(\sqrt{N})$ worst case	$O(P)$ after build
Precompute	none	$O(\text{range} \times S)$ once
Memory	$O(1)$	$O(P)$
Big semiprime, no small factor	correct but slow ($\sqrt{N}$)	returns [] (misses)
Best use	one-shot smallest-factor engine	screening many N for small factors

On the shared test numbers.

N	wheel_scan	sieve N_P
4126375141919523791919279	3 (instant)	[148021] (misses 3)
4126375141919523791919271	7 (instant)	[7, 23, 47, 161, 329, 1081, 7567]
7334938410324762341973827534	2521 (instant)	[2521]
419726354193798261		
412637514191952379171729	too slow ($\sim 4 \times 10^8$ steps)	[] (no small factor)

Note the two disagreements. For 4126375141919523791919279, **wheel_scan** returns 3 (it peels 3 first), while the sieve misses 3 entirely and reports 148021 — because P was built skipping multiples of 3. For 4126375141919523791919271 the sieve lists composite divisors ($161 = 7 \cdot 23$, $329 = 7 \cdot 47$, $1081 = 23 \cdot 47$, $7567 = 7 \cdot 23 \cdot 47$) alongside the primes; **wheel_scan** returns just the smallest prime, 7.

4 Conclusion

The two methods are complementary faces of the same $6k \pm 1$ idea:

- **wheel_scan** is a *complete, one-shot* smallest-factor finder: correct for every N , instant when a small factor exists, and $O(\sqrt{N})$ in the worst case.
- the *sieve* is a *reusable, partial* screen: cheap per query after one build, ideal for testing many numbers for *small* factors, but blind to factors larger than its range (it returns [] on 412637514191952379171729, whose factors are $\sim 2.5 \times 10^9$ and $\sim 1.6 \times 10^{14}$).

Neither escapes the $\sqrt{N}$ wall: the sieve is fast only because it does *less* (small factors only), and **wheel_scan** is exactly 2,3-peeled trial division. For semiprimes with two large prime factors, both give way to Pollard's rho / $p - 1$ or ECM. The natural pipeline is: **sieve or wheel first** (peel small factors cheaply), **then Pollard** for the large cofactor.